

Course: BCS407 – Artificial Intelligence

Model Improvement and Research

Subtitle: AI-Based Smart Campus Detection System – Advanced Iteration

Theme 3: Campus Safety Monitoring (Wet Floor Signs, Fire Alarms, Emergency Exits, Safety Equipment)

Name	ID
Hassan Mujtaba	20220002085
Izaan Shaikh	20220001735
Khaled Riyan	20220001526
Leanne Jessica Rodrigo	20210001983

Instructor: Dr Najla Al Futaisi

Date of Submission: 13th May 2026

Abstract

We improved the Project 1 AI-based campus safety detection system by fixing the evaluation setup and improving weak class detection. The Project 1 model detected 15 safety classes, including fire alarms, emergency exits, blocked exits, helmets, heads, people, and warning signs. However, the original configuration had a major weakness because training and validation data overlapped, which could make the baseline results look stronger than real-world performance. For Project 2, we used two improvement strategies. First, we corrected the train/validation/test split, upgraded the model from YOLOv8m to YOLOv8l, and used AdamW for more stable training. Second, we expanded the dataset for under-represented safety classes to reduce class imbalance. Experiment 1 achieved Precision 0.441, Recall 0.599, mAP@0.5 0.472, and mAP@0.5:0.95 0.300. Experiment 2 became the final combined model because it used the corrected split, YOLOv8l, AdamW, and the expanded dataset. Its summary results were Precision 0.8405, Recall 0.6213, mAP@0.5 0.4975, and mAP@0.5:0.95 0.3275. The final scores are lower than the original baseline, but they are more honest and useful because they are based on a stricter evaluation setup.

Keywords

Keywords: Object Detection, YOLOv8l, Campus Safety Monitoring, Dataset Expansion, Class Balancing, Artificial Intelligence, Real-Time Detection

1. Introduction

Campus safety is important because hazards such as blocked exits, missing warning signs, and fire alarm issues can affect emergency response. Manual checking through guards or CCTV alone is not enough for a busy university campus. Because of that, we used computer vision to help detect safety objects and possible violations automatically.

In Project 1, we built a smart campus safety detection system using YOLOv8m. The model was trained to detect 15 classes, including fire alarms, exit signs, blocked exits, helmets, heads, persons, and general safety signs.

The first model had strong headline scores, but it also had clear weaknesses. The biggest issue was that the original data configuration allowed overlap between training and validation data. This can make results look better than they really are. The dataset was also imbalanced, with many examples for common classes and very few for some safety-critical classes.

In Project 2, we focused on two improvements. First, we corrected the training pipeline by using a clean data split, YOLOv8l, and AdamW. Second, we expanded the dataset for weaker classes so the model had more examples of rare safety situations.

We evaluated all runs using Precision, Recall, $mAP@0.5$, $mAP@0.5:0.95$, confusion matrix analysis, and per-class results.

These are the questions that this project will address:

1. How will a train, validation and test split affect the reliability of object detection evaluation?
2. Does dataset expansion and class balancing improve detection of minority and underperforming classes?
3. What are the most effective improvement measures that improve the overall performance and robustness of the campus safety detection system?

2. Related Work

2.1 Object Detection and YOLO Models

Object detection finds both what an object is and where it appears in an image. Older methods such as HOG and region-based CNNs improved visual recognition, but they were not ideal for real-time monitoring because they were heavier to run (Dalal & Triggs, 2005; Girshick et al., 2014). Faster R-CNN improved region proposal learning, while SSD and YOLO made detection faster by predicting boxes and classes in one pass (Ren et al., 2015; Liu et al., 2016; Redmon et al., 2016).

We used YOLOv8 because it supports real-time detection, transfer learning, and different model sizes. YOLOv8l gives more feature capacity than smaller models, which helps with small signs and similar-looking safety objects. The trade-off is that it is heavier, so hardware still matters (Jocher et al., 2023).

2.2 Application-Specific Safety Detection Studies

Safety monitoring is a good computer vision use case because many hazards are visible, such as missing PPE, unsafe work areas, and blocked access routes. Nath et al. (2020) used YOLO-based models for real-time PPE checking, while Han et al. (2023) improved helmet detection with super-resolution and YOLOv5. These studies support our choice of object detection for campus safety, but our work is different because we handle mixed campus classes, not only PPE.

2.3 Dataset Imbalance and Augmentation

Class imbalance is a major issue because rare classes can still be safety-critical. A model may score well overall while missing the rare hazards that matter most. Buda et al. (2018) discuss how imbalance affects deep learning, and Oksuz et al. (2021) review imbalance problems in object detection. Crasto (2024) also studied class imbalance in single-stage detectors and found that mosaic and mixup style augmentation can help minority-class robustness. This supports our decision to look beyond one overall mAP score and review per-class results.

We used dataset expansion because augmentation alone cannot replace real examples of rare classes. Flips, scaling, colour variation, and mosaic training help the model see more variety, but real blocked-exit and exit-related images are still needed for better learning. Zoph et al. (2020) show that augmentation can improve object detection performance, while Loshchilov and Hutter (2019) support AdamW as a stronger optimization choice through decoupled weight decay.

2.4 Real-World Detection Challenges

For real use, the model must handle more than clean validation images. CCTV footage can include low light, motion blur, high camera angles, occlusion, clutter, and domain shift. This is why we treated the Project 1 baseline critically and used a cleaner split in Project 2.

3. Baseline Model Analysis

3.1 Original Dataset

We used the Project 1 campus safety detector as our baseline. Its purpose was to identify safety objects and hazards in campus-style scenes.

The original dataset was prepared in Roboflow and contained 1,672 images across 15 safety classes. These classes included fire alarms, exit directions, blocked exits, emergency exit doors, exit signs, helmets, heads, persons, and general signs. The dataset was exported in YOLOv8 format.

The images covered different lighting conditions, scenes, distances, and camera perspectives, which helped make the task closer to real use.

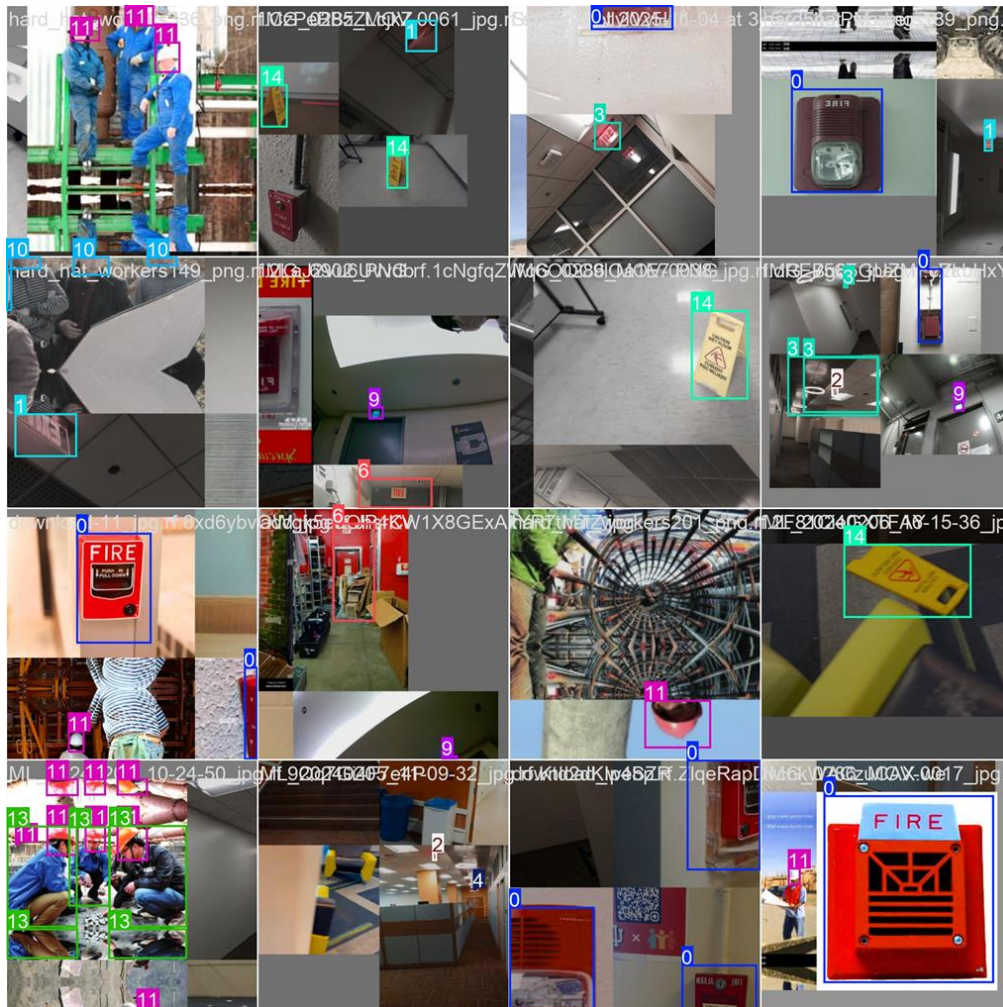


Figure 1: Training batch 0 - annotated samples with bounding boxes around safety-related classes (helmets, fire alarms, persons, exit signs).



Figure 2: Training batch 1 - additional annotated samples showing varied indoor/outdoor scenes.

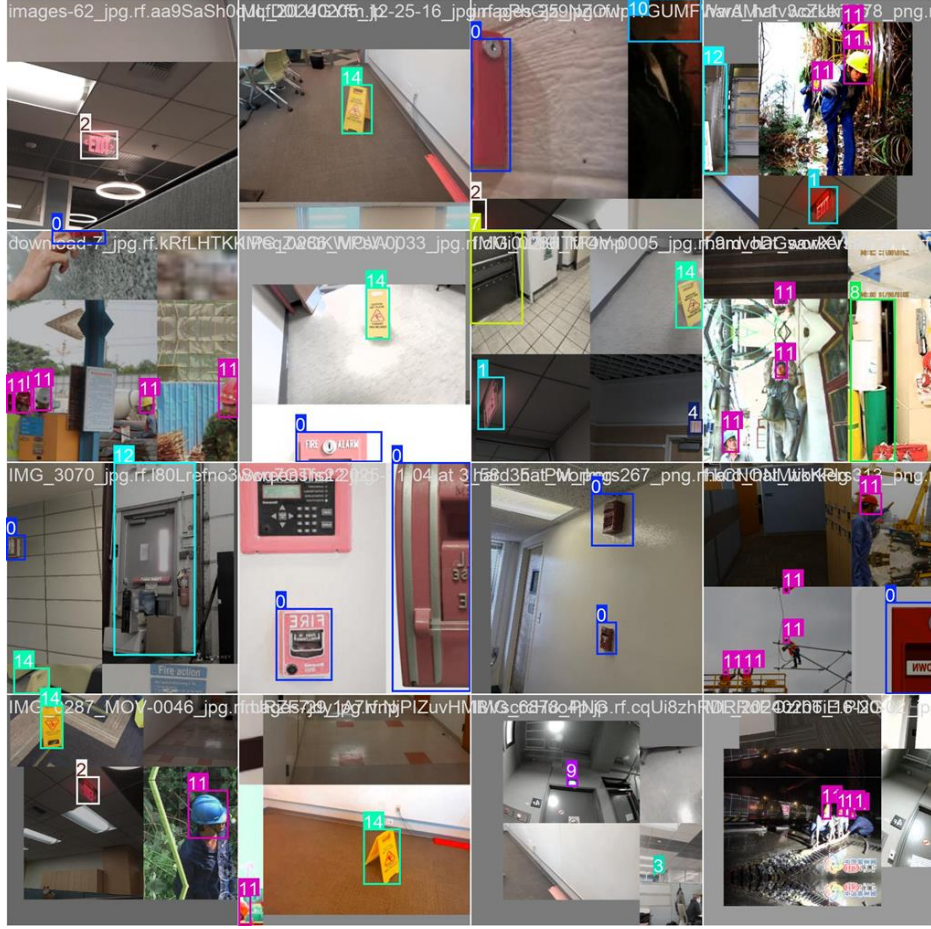


Figure 3: Training batch 2 - annotated samples featuring different lighting conditions and perspectives.

We used this dataset as the base before creating the cleaned and expanded Project 2 versions.

3.2 Dataset Split and Class Distribution

The cleaned Project 2 split shown in our experiment output contained 2,167 images: 1,386 for training, 396 for validation, and 377 for testing. The table reports the per-class instance distribution across those splits.

Table 1: Class distribution by train, validation, and test split for the cleaned Project 2 dataset.

Class	Train	Validation	Test	Total
Fire alarm	271	93	60	424
Left Exit	119	33	12	164
Left/Right Exit	157	50	21	228
Right Exit	239	58	33	330
Straight Exit	122	35	23	180
blocked emergency exit	15	4	1	20
boxes blocking exit	10	3	0	13
emergency exit door	10	1	3	14
exit block violation	2	1	2	5
exit-sign	49	10	4	63
head	364	109	70	543
helmet	1241	404	223	1868
objects blocked exit	14	3	1	18
person	19	5	4	28
sign	156	36	20	212

3.3 Original YOLOv8m Configuration

The Project 1 baseline used YOLOv8m. We kept it as the official baseline because it balanced speed and accuracy for the first version of the system.

Using pretrained yolov8m.pt weights helped the model start from useful general visual features instead of learning everything from scratch.

It was trained with the following setup:

Parameter	Value
Model Architecture	YOLOv8m
Pretrained Weights	yolov8m.pt
Epochs	50
Batch Size	32
Image Size	640 × 640
Framework	Ultralytics YOLOv8
Annotation Format	YOLO format

Table 2: Baseline YOLOv8m Training Configuration Used in Project 1.

Ultralytics training included standard augmentations such as mosaic, flipping, and scaling.

Project 1 reported Precision, Recall, F1, mAP@0.5, and confusion matrix results.



Figure 4: YOLOv8m Model Pipeline - from dataset preparation through annotation, training, evaluation, and inference.

3.4 Baseline Results

The Project 1 baseline performed well on paper. We carried forward its Precision, Recall, mAP@0.5, mAP@0.5:0.95, and confusion matrix analysis for comparison.

The baseline reached mAP@0.5 = 0.978. Common classes such as helmets, persons, and exit signs performed strongly, but the result needed caution because of the validation overlap.

Project 1 also produced training curves, confusion matrices, Precision-Recall curves, F1-confidence curves, and validation predictions.

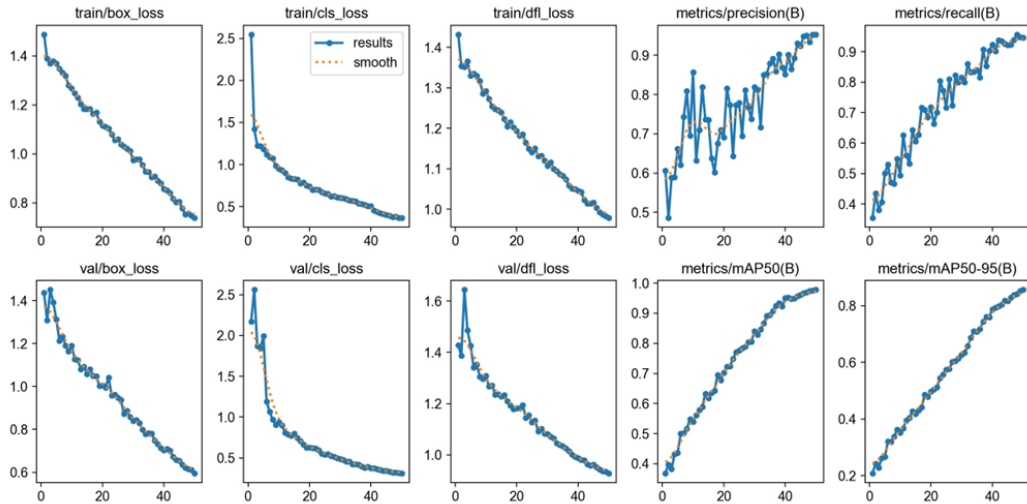


Figure 5: Training and validation losses across 50 epochs - bounding box, classification, and DFL losses all decrease consistently.

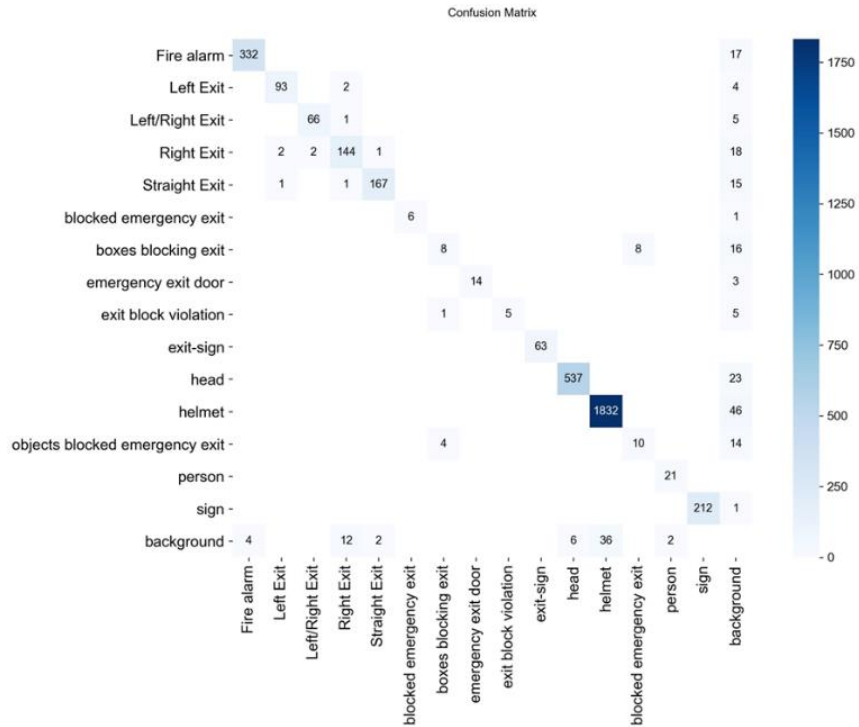


Figure 6: Confusion Matrix - actual counts of true versus predicted class labels for all 15 categories.

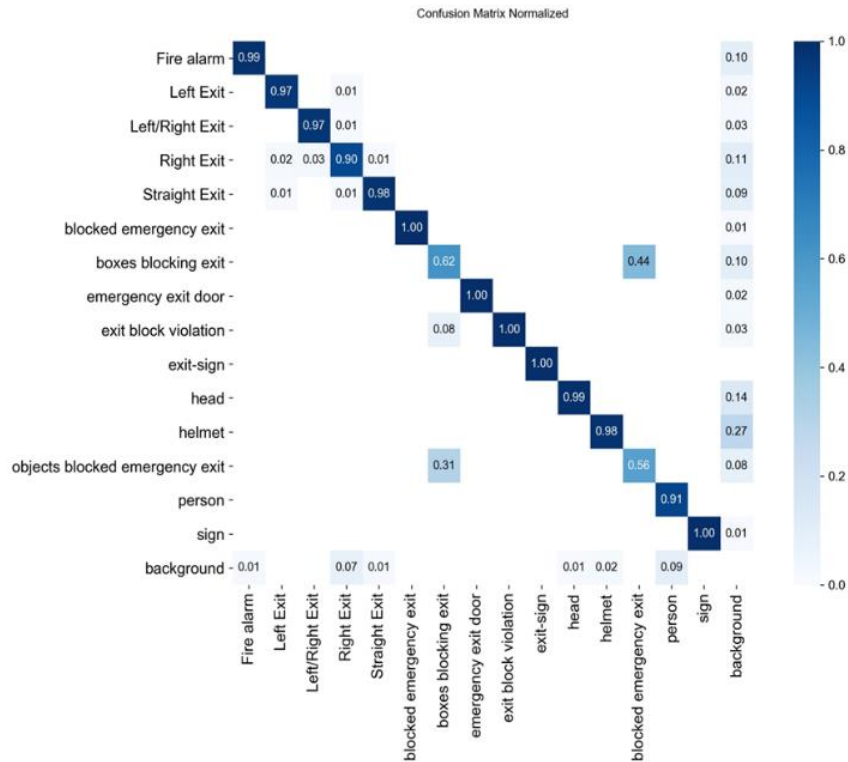


Figure 7: Normalized Confusion Matrix - relative precision per category; most classes score between 0.90 and 1.00 on the diagonal.

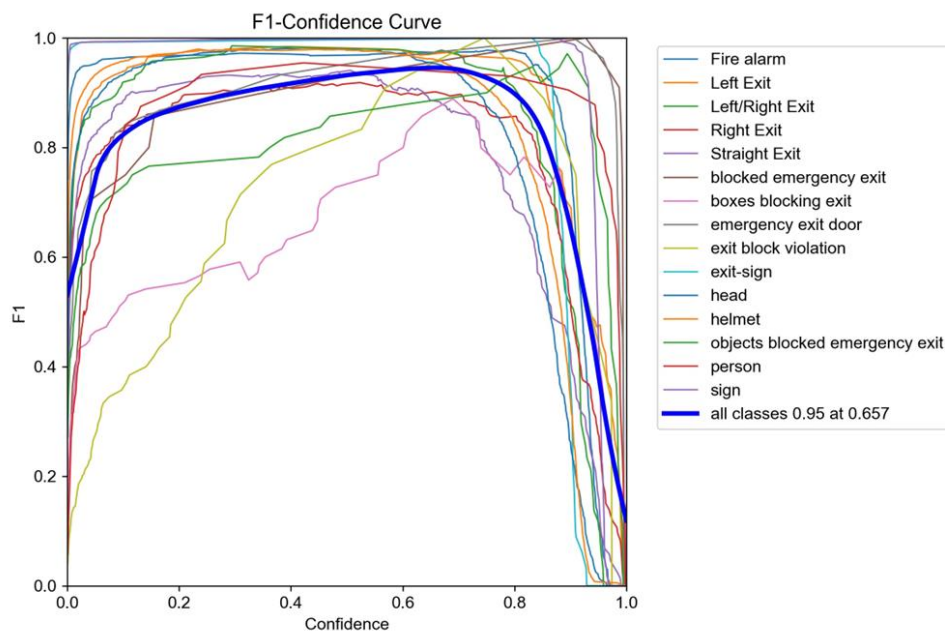


Figure 8: F1-Confidence Curve - peak F1 score of 0.95 achieved at confidence threshold 0.657 across all 15 classes.

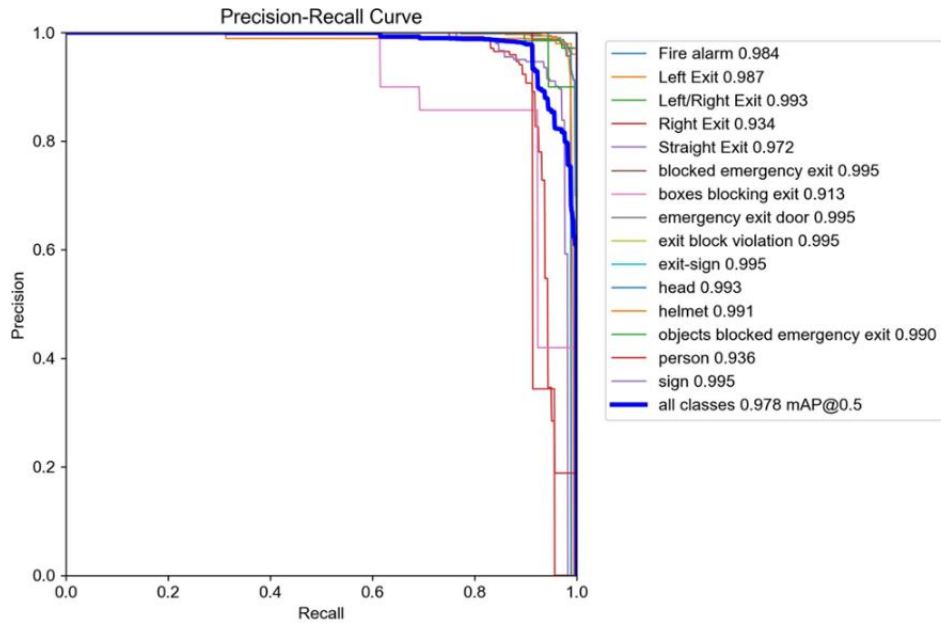


Figure 9: Precision-Recall Curve - overall mAP@0.5 = 0.978; most safety-critical classes exceed AP 0.99.

These results looked strong, but they also showed why Project 2 was needed: evaluation reliability, class imbalance, and minority-class weakness were not fully solved.

3.5 Baseline Weakness Analysis

The baseline had good overall scores, but we found problems that made it less reliable as a real safety system.

The main issue was evaluation reliability. Since the original training and validation sets overlapped, the reported scores could be too optimistic and may not show true performance on unseen images.

A second issue was class imbalance. Common classes such as helmet and person had far more examples than blocked-exit and exit-violation classes, so the model had less chance to learn rare hazards properly.

A third issue was label similarity. Several exit-obstruction classes look very similar, so the model sometimes confused them.

We also noticed signs of overfitting, because long training gave limited extra improvement after a certain point. This pushed us to use a more controlled setup in Project 2.

4. Methodology

4.1 Experimental Design

In Project 2, we improved the baseline through controlled experiments. We first fixed the evaluation setup, then tested model and dataset changes, and finally compared the results against the Project 1 baseline.

All experiments were compared using the same main metrics: Precision, Recall, mAP@0.5, mAP@0.5:0.95, and confusion matrix analysis.

Two strategies for improving the object detection campus safety model can be summarized as follows:

- Improvements to the training pipeline and controlled training.

- Improvements to the dataset used in training.

4.2 Experiment 1 - Architecture and Evaluation Optimization

Experiment 1 focused on making the training and evaluation process more reliable. We used a clean train, validation, and test split so the validation results were not based on images seen during training.

We also moved from YOLOv8m to YOLOv8l to test whether a larger backbone could learn better features for complex campus safety scenes.

The clean split was necessary before judging any improvement, because the old validation setup was not independent.

We used AdamW because it applies decoupled weight decay and is widely used to improve training stability in deep learning. In our case, AdamW was used with the cleaner split and YOLOv8l so the training process was more controlled.

Training was kept controlled instead of simply running longer and risking memorisation.

4.3 Experiment 2 - Dataset Expansion and Class Balancing

Experiment 2 focused on the data. We added more images for underrepresented safety classes because the baseline struggled most with rare and visually similar hazards.

We focused especially on emergency-exit and blocked-exit classes, because these are important for safety but were weak in the baseline.

The expanded dataset also kept YOLO augmentations such as mosaic, scaling, and flipping to improve variation during training.

This gave the model more real examples and more augmented variations for difficult classes.

4.4 Final Combined Model

The final model used the successful setup from Experiment 2: YOLOv8l, the corrected split, AdamW, and the expanded dataset. We treat Experiment 2 as the final combined model because it already combines the successful improvements.

This avoids pretending there was a separate final run when the combined run is the actual final model.

We compared the results against:

Original Baseline Model

Experiment 1

Experiment 2

Final Combined Model

This comparison helped us show what changed from the baseline to the corrected and expanded Project 2 model.

4.5 Training Configuration

We ran the experiments in Python using Ultralytics YOLOv8, pretrained YOLOv8l weights, and YOLO-format labels from Roboflow.

The main training configuration included:

Parameter	Configuration
Model Architecture	YOLOv8l
Framework	Ultralytics YOLOv8
Annotation Format	YOLO format
Input Resolution	640 × 640
Transfer Learning	Pretrained yolov8l.pt weights
Optimizer	AdamW
Learning Rate	0.001
Batch Size	8
Augmentation	Mosaic augmentation, scaling, and random flipping

Table 3: Final YOLOv8l Training Configuration for Improved Model Experiments.

4.5.1 YOLOv8 Model Size Comparison

We selected YOLOv8l because the weak classes involved small objects and visually similar signs or obstructions. It is stronger than YOLOv8m, but it also needs more computation.

Model	Parameters	FLOPs @640	Reason in this project
YOLOv8n	3.2M	8.7B	Fastest, but too small for complex 15-class detection
YOLOv8s	11.2M	28.6B	Good speed, lower feature capacity
YOLOv8m	25.9M	78.9B	Balanced baseline-level option
YOLOv8l	43.7M	165.2B	Chosen final model for stronger feature extraction
YOLOv8x	68.2M	257.8B	Highest capacity, heavier than needed for this project

Table 4: YOLOv8 variant comparison showing why YOLOv8l was selected for the improved model.

We evaluated each run with Precision, Recall, $\text{mAP}@0.5$, $\text{mAP}@0.5:0.95$, confusion matrices, PR curves, and validation prediction images.

The model is a research prototype for CCTV-based campus safety monitoring, not a fully used system yet.

5. Results

5.1 Baseline Results

The Project 1 baseline used YOLOv8m and reported strong results across most classes. It reached Precision 0.953, Recall 0.946, $\text{mAP}@0.5$ 0.978, and $\text{mAP}@0.5:0.95$ 0.856. These values were high, especially for common classes such as helmets, persons, warning signs, and exit signs.

However, we do not treat those numbers as fully reliable because the training and validation overlap could have inflated performance. Minority safety classes also showed more confusion than the overall mAP suggested.

5.2 Experiment 1 Results

In Experiment 1, we used a corrected split, YOLOv8l, and AdamW. The scores dropped compared with Project 1, but that was expected because the validation was now more honest. Experiment 1 achieved Precision 0.441, Recall 0.599, $\text{mAP}@0.5$ 0.472, and $\text{mAP}@0.5:0.95$ 0.300, with the best epoch at 37.

5.3 Experiment 2 Results

Experiment 2 added more samples for weak safety classes and used YOLOv8l, AdamW, seed 42, batch size 8, and 50 epochs. The training summary reported best Precision 0.8405, best Recall 0.6213, overall $\text{mAP}@0.5$ 0.4975, and overall $\text{mAP}@0.5:0.95$ 0.3275. This improved mAP and recall compared with Experiment 1, but the results still show that minority classes need more data and cleaner labels.

5.4 Final Model Comparison

Table 4 compares the Project 1 baseline, Experiment 1, and Experiment 2. We label Experiment 2 as the final combined model because it includes the corrected split, YOLOv8l, AdamW, and the expanded minority-class dataset. This makes the table transparent instead of adding a duplicate final run.

Table 5: Conclusive Comparison of Baseline, Experiment 1, Experiment 2, and Final Model.

Parameter	Baseline (P1)	Exp 1: YOLOv8l	Exp 2: Expanded Dataset	Final Model
Model	YOLOv8m	YOLOv8l	YOLOv8l	YOLOv8l
Epochs	50	50	50	50
Batch Size	32	16	8	8
Learning Rate	0.01	0.001	0.001	0.001
Dataset / split	1,672 total; old val overlap	1,386 train / 396 val / 377 test	expanded dataset	same as Exp 2
Optimizer	Auto (SGD)	AdamW	AdamW	AdamW
Seed	-	42	42	42
Best Epoch	-	37	50	50
Precision	0.953	0.441	0.8405	0.8405
Recall	0.946	0.599	0.6213	0.6213
mAP@0.5	0.978	0.472	0.4975	0.4975
mAP@0.5:0.95	0.856	0.300	0.3275	0.3275
F1 Peak	Not reported	0.45	0.622	0.622

5.5 Per-Class Analysis

We reviewed per-class behaviour to see which classes were still weak. Common classes such as helmets, persons, and general signs were easier for the model than rare exit-obstruction classes. The biggest problems were still around blocked emergency exits and objects near exit areas. These classes look similar and often appear in cluttered scenes. Dataset expansion helped the model see more examples of weak classes, but it did not completely solve the problem. More targeted images and cleaner labels are still needed.



Figure 10: Dataset Analysis and Per-Class Final Results - class distribution across train/valid/test splits, per-class AP@0.5, precision, and recall for the improved YOLOv8l campus safety model.

5.6 Error Analysis

We checked the main error cases from the validation outputs. Most mistakes came from similar-looking classes, partial occlusion, low-quality images, and cluttered backgrounds. The most common confusion was between blocked exits and objects placed near exit areas. The model sometimes used scene context instead of the exact object boundary. The image below is a clear example of the model predicting fire alarm with the confidence of 0.68 this is class confusion error the model is detecting the wrong class due to the object being red colour and attached to the wall lastly the exit sign is not detected those two errors are perfect example that training needs more diverse and much larger dataset for training the dataset is

very limited of about 2000 pictures



False positives could create unnecessary alerts, while false negatives could miss real hazards. This is why the system should support human review.

This error analysis shows that the next improvement should be better data, clearer labels, and testing on real campus CCTV footage.

6. Discussion

Our main finding is that the Project 1 baseline was too optimistic because training and validation data overlapped. After using a clean split, the metrics dropped, but the new numbers are more believable.

Experiment 1 improved evaluation validity, but it also bundled YOLOv8l, AdamW, and the clean split together. This is a limitation because the rubric prefers isolated experiments. We bundled them because the clean split was not an optional improvement; it was needed before any fair evaluation could happen.

The final model is still not perfect. Blocked exits, emergency exit doors, and exit-block violations remain difficult because there are fewer examples and the labels overlap visually. Some categories may need to be merged or redefined before use.

Overall, Project 2 makes the system more honest and research-ready. The final scores are lower than Project 1, but the evaluation is stronger, the dataset weakness is clearer, and the next steps are easier to justify.

7. Ethical Reflection

7.1 Privacy and Surveillance Considerations

The system is designed for safety monitoring, but it could raise privacy concerns if connected to CCTV. We should avoid identifiable faces, ID numbers, licence plates, and other personal information. In use, access to footage and predictions must be limited to authorised safety staff.

7.2 Bias, Fairness, and Dataset Representation

Fairness means the model should not work well only on common classes while failing on rare but important hazards. Class imbalance is therefore both a technical issue and an ethical issue. If the system misses blocked exits, people could be put at risk.

7.3 Reliability and Safety Risks

False positives can create unnecessary alerts, while false negatives can miss real hazards. For that reason, the model should not punish or blame individuals automatically. Staff should review alerts before action is taken.

7.4 AI Ethics Framework

We link our ethics approach to the OECD AI Principles: robustness, transparency, accountability, and responsible use. Robustness was addressed through cleaner evaluation and dataset expansion. Transparency is supported through bounding boxes, labels, confidence scores, and reported limitations. Accountability means humans remain responsible for safety decisions.

8. Real-World Use

8.1 Proposed Use Architecture

A realistic use would connect campus CCTV streams to an edge GPU or local inference server running the final YOLOv8l model. The system would detect fire alarms, exits, helmets, persons, warning signs, and blocked-exit hazards, then send alerts to a dashboard.

8.2 Impact of the Improved Model on Use

The corrected split makes use planning more realistic because the final metrics are based on unseen validation and test data. Dataset expansion also gives the model more exposure to weak safety classes.

8.3 Remaining Use Challenges

The main use challenges are domain shift, low-light CCTV footage, motion blur, occlusion, and false alerts. We should test the model on real campus camera footage before full use and tune thresholds carefully.

8.4 Future Use Improvements

Future work should collect more real campus images, add rare-class samples, tune confidence and NMS thresholds, test multi-scale training, and use tracking so alerts trigger only when a hazard stays visible for a few seconds.

9. Conclusion

In Project 2, we improved the campus safety detector by fixing the dataset split, moving to YOLOv8l, using AdamW, and expanding weak classes. The results show that the old baseline was stronger on paper, but the new evaluation is more realistic.

The most important outcome is not just a higher score. We now have a more honest view of the model's strengths and weaknesses. Dataset expansion helped compared with Experiment 1, but rare and visually similar classes still need more data, cleaner labels, and real CCTV testing.

Project Asset Links

Final model weights:

- Exp 1 best.pt: <https://huggingface.co/Hassanmujtabat/campus-safety-detection-exp1/blob/main/best.pt>
- Exp 2 best.pt (Final Model): <https://huggingface.co/Hassanmujtabat/campus-safety-detection-exp2/blob/main/best.pt>

Datasets:

- Original dataset (Exp 1): <https://huggingface.co/Hassanmujtabat/campus-safety-detection-exp1/blob/main/safety%20helmet.yolov8.zip>
- Additional dataset added for Exp 2: <https://huggingface.co/Hassanmujtabat/campus-safety-detection-exp2/blob/main/Emergency%20Exit%20Signs.yolov8.zip>
- Full combined dataset (final): <https://huggingface.co/Hassanmujtabat/campus-safety-detection-exp2/blob/main/train.rar>

- Pre-split dataset: <https://huggingface.co/Hassanmujtabat/campus-safety-detection-exp2/blob/main/train%20split.zip>

Code repository: <https://github.com/Hassan-Mujtabata/BCS407-Campus-Safety-Detection-P2>

AI Assistance Declaration

AI writing assistance was used for proofreading, structure, and wording refinement only. All experimental results, figures, dataset outputs, model metrics, and final interpretations were produced and reviewed by the group.

References

- Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106, 249-259.
<https://doi.org/10.1016/j.neunet.2018.07.011>
- Crasto, N. (2024). Class imbalance in object detection: An experimental diagnosis and study of mitigation strategies. *arXiv*. <https://arxiv.org/abs/2403.07113>
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 1, 886-893. <https://doi.org/10.1109/CVPR.2005.177>
- Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 580-587. <https://doi.org/10.1109/CVPR.2014.81>
- Han, J., Liu, Y., Li, Z., Liu, Y., & Zhan, B. (2023). Safety helmet detection based on YOLOv5 driven by super-resolution reconstruction. *Sensors*, 23(4), Article 1822.
<https://doi.org/10.3390/s23041822>
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. *Ultralytics*.
<https://docs.ultralytics.com/models/yolov8/>
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision*, 2980-2988.
<https://doi.org/10.1109/ICCV.2017.324>
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. *Computer Vision - ECCV 2016*, 21-37.
https://doi.org/10.1007/978-3-319-46448-0_2
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *Proceedings of the 7th International Conference on Learning Representations*.

- Nath, N. D., Behzadan, A. H., & Paal, S. G. (2020). Deep learning for site safety: Real-time detection of personal protective equipment. *Automation in Construction*, 112, Article 103085. <https://doi.org/10.1016/j.autcon.2020.103085>
- Oksuz, K., Cam, B. C., Kalkan, S., & Akbas, E. (2021). Imbalance problems in object detection: A review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(10), 3388-3415. <https://doi.org/10.1109/TPAMI.2020.2981890>
- Organisation for Economic Co-operation and Development. (2019). OECD principles on artificial intelligence. <https://www.oecd.org/en/topics/ai-principles.html>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779-788. <https://doi.org/10.1109/CVPR.2016.91>
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28.
- Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6, Article 60. <https://doi.org/10.1186/s40537-019-0197-0>
- Zoph, B., Cubuk, E. D., Ghiasi, G., Lin, T.-Y., Shlens, J., & Le, Q. V. (2020). Learning data augmentation strategies for object detection. *Computer Vision - ECCV 2020*, 566-583. https://doi.org/10.1007/978-3-030-58583-9_34

Appendix

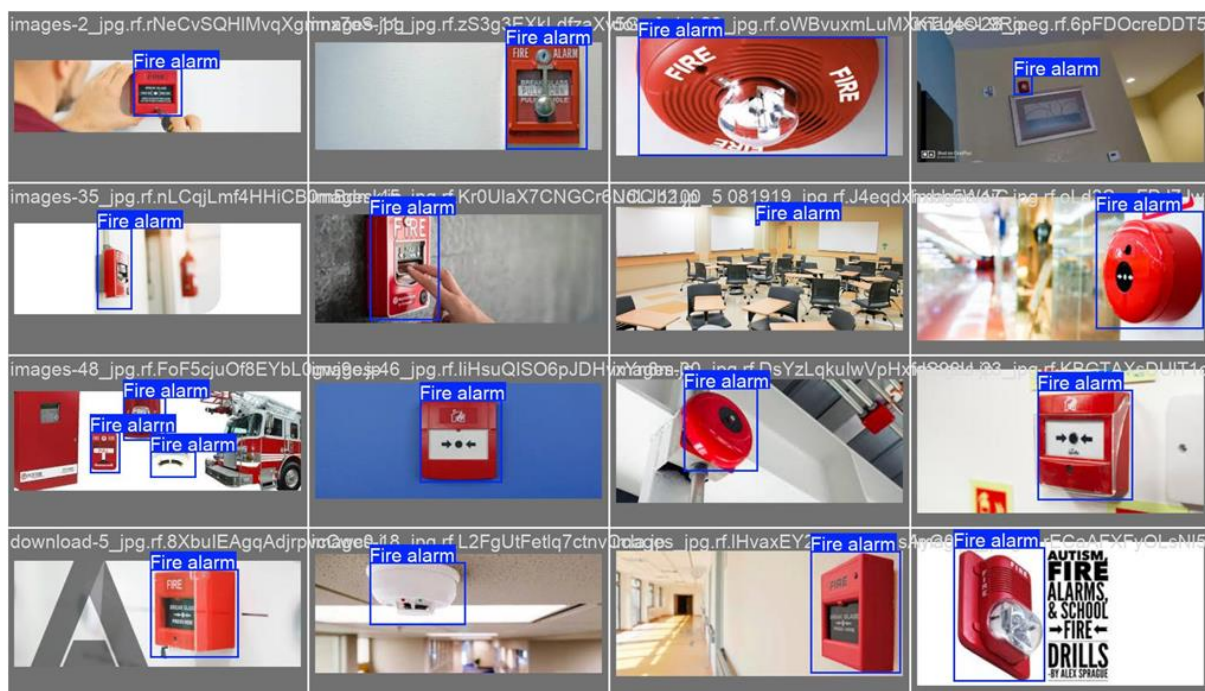


Figure A1: Validation batch 0 - ground truth labels for all 15 safety object classes.

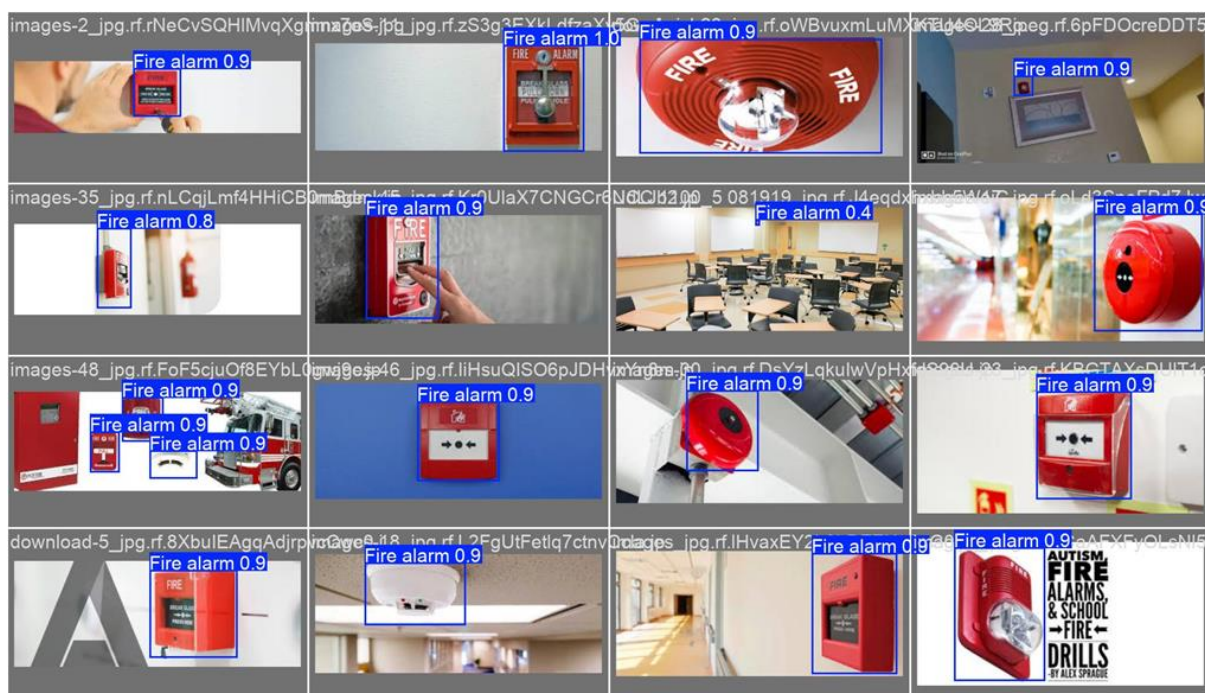


Figure A2: Validation batch 0 - model predictions with bounding boxes and confidence scores closely matching ground truth labels.

